

Tu es chargé de concevoir, développer, tester, sécuriser et documenter intégralement une application web internationale de marketplace de prize competitions, free draws et sweepstakes.

Tu dois agir simultanément comme :

- architecte logiciel senior ;
- ingénieur full-stack principal ;
- expert PostgreSQL et Supabase ;
- product designer senior ;
- spécialiste sécurité applicative ;
- ingénieur QA ;
- ingénieur DevOps ;
- responsable de la documentation technique.

Le nom provisoire du produit est « ChanceMarket ». Centralise le nom, le logo textuel, les métadonnées et les couleurs dans une configuration afin qu'ils puissent être remplacés facilement.

Ne produis pas uniquement une maquette, une landing page ou un prototype visuel. Développe une véritable application fonctionnelle, connectée à sa base de données, avec authentification, stockage, paiements configurables, administration, tests, migrations et documentation de déploiement.

1. Vision du produit

ChanceMarket est une marketplace internationale sur laquelle des vendeurs approuvés peuvent proposer des biens dans le cadre de campagnes promotionnelles.

Exemples de lots :

- vélos ;
- smartphones ;
- ordinateurs ;
- consoles ;
- montres ;
- objets de collection ;
- mobilier ;
- véhicules ;
- biens immobiliers, uniquement lorsque la juridiction, le prestataire de paiement, les assurances et les procédures opérationnelles le permettent ;
- autres catégories autorisées par la plateforme.

Selon la juridiction et la configuration de la campagne, un utilisateur peut participer par :

- une entrée payante ;
- une voie de participation gratuite alternative ;
- une prize competition avec une véritable épreuve d'habileté, de connaissance ou de jugement ;
- un sweepstake sans achat obligatoire ;
- un autre mécanisme expressément activé par l'équipe conformité.

À la clôture de la campagne, un gagnant éligible est déterminé selon les règles officielles de la campagne et un processus serveur auditable.

Le vendeur reçoit le montant contractuellement prévu, après déduction des frais, réserves, remboursements, taxes, coûts de livraison et commissions applicables.

2. Principes non négociables

Respecte impérativement les principes suivants :

1. Aucun format de campagne ne doit être activé mondialement par défaut.
2. L'éligibilité doit dépendre du pays, de la subdivision administrative, de l'âge, du type de campagne, du lot, du vendeur et des règles actives.
3. Les règles territoriales doivent être stockées en base et administrables sans redéploiement.
4. Une campagne ne peut être publiée que si sa configuration est autorisée dans au moins une juridiction active.
5. Un utilisateur ne peut participer que si son territoire est autorisé.
6. Le navigateur ne doit jamais décider seul de l'éligibilité.
7. Les contrôles critiques doivent être répétés côté serveur et en base de données.
8. Aucun secret, aucune clé privilégiée et aucune logique de tirage ne doivent être exposés au client.
9. Les paiements doivent être traités par un prestataire ayant explicitement approuvé l'activité et les juridictions concernées.
10. N'intègre pas un prestataire de paiement en supposant que ses conditions autorisent cette activité.
11. Utilise une architecture modulaire permettant de changer de prestataire.
12. Les mentions juridiques générées dans le projet sont des modèles provisoires devant être remplacés ou approuvés par des juristes locaux.
13. Ne présente jamais l'application comme juridiquement conforme par défaut.
14. Ne contourne pas les restrictions d'un prestataire de paiement, d'un pays ou d'un État.
15. Les fonctions financières, territoriales et de sélection doivent être transactionnelles, idempotentes et auditable.

3. Résultat attendu

À la fin de ton travail, le dépôt doit contenir :

- l'application Next.js complète ;
- le frontend responsive ;
- le backend Supabase ;
- toutes les migrations SQL ;
- toutes les politiques Row Level Security ;
- les fonctions PostgreSQL nécessaires ;
- les Supabase Edge Functions nécessaires ;
- le système d'authentification ;
- le stockage des images et documents ;
- les interfaces de paiement et de remboursement ;
- le moteur de conformité territoriale ;
- le système de participations ;
- le mécanisme de sélection du gagnant ;
- le tableau de bord vendeur ;
- le back-office administrateur ;
- les notifications ;

- les emails transactionnels sous forme de templates ;
- les tests unitaires, d'intégration et end-to-end ;
- les données de démonstration ;
- le pipeline CI ;
- la documentation d'installation ;
- la documentation de sécurité ;
- les instructions de déploiement ;
- un fichier `.env.example` complet ;
- un inventaire clair de tout ce qui nécessite encore une validation juridique, commerciale ou opérationnelle.

Il ne doit rester aucun TODO sur les parcours principaux.

4. Méthode d'exécution

Travaille en phases et maintiens un fichier `IMPLEMENTATION_STATUS.md`.

Avant de coder :

1. inspecte le dépôt ;
2. identifie les fichiers existants ;
3. rédige l'architecture proposée ;
4. rédige le modèle de données ;
5. rédige le modèle de sécurité ;
6. rédige le plan d'implémentation ;
7. commence ensuite le développement.

Après chaque phase importante :

- lance le typage TypeScript ;
- lance ESLint ;
- lance les tests pertinents ;
- lance le build de production ;
- corrige les erreurs avant de poursuivre ;
- actualise `IMPLEMENTATION_STATUS.md`.

Ne demande pas une validation humaine pour chaque décision mineure. Prends des décisions techniques cohérentes, documente-les et avance.

Ne simule jamais l'exécution d'une commande. Exécute réellement les commandes disponibles et rapporte honnêtement les résultats.

5. Stack technique imposée

Utilise :

- Next.js dans sa version stable actuelle ;
- App Router ;
- React ;
- TypeScript avec mode strict ;
- Tailwind CSS ;

- shadcn/ui fortement personnalisé ;
- Framer Motion pour les animations utiles ;
- React Hook Form ;
- Zod ;
- TanStack Query uniquement pour les flux client qui le justifient ;
- Server Components par défaut ;
- Server Actions ou Route Handlers lorsque pertinent ;
- Supabase ;
- PostgreSQL Supabase ;
- Supabase Auth ;
- Supabase Storage ;
- Supabase Realtime ;
- Supabase Edge Functions ;
- Supabase CLI ;
- Vitest ;
- React Testing Library ;
- Playwright ;
- ESLint ;
- Prettier ;
- GitHub Actions.

Déploiement recommandé :

- Next.js sur Vercel ;
- PostgreSQL, Auth, Storage, Realtime et Edge Functions sur Supabase ;
- fournisseur d'emails transactionnels derrière une interface ;
- fournisseur de paiement approuvé derrière une interface ;
- service de géolocalisation derrière une interface ;
- service KYC/AML derrière une interface.

Utilise `pnpm` .

6. Architecture logicielle

Organise le projet par domaines fonctionnels.

Structure recommandée :

```
app/
  (marketing)/
  (auth)/
  (dashboard)/
  admin/
  api/

components/
  ui/
  layout/
  marketing/
  campaigns/
```

```
payments/  
compliance/  
forms/  
feedback/  
  
features/  
  auth/  
  profiles/  
  sellers/  
  campaigns/  
  entries/  
  payments/  
  draws/  
  compliance/  
  geolocation/  
  verification/  
  moderation/  
  notifications/  
  disputes/  
  admin/  
  analytics/  
  
lib/  
  supabase/  
  auth/  
  payments/  
  compliance/  
  security/  
  validation/  
  money/  
  dates/  
  localization/  
  rate-limit/  
  audit/  
  errors/  
  observability/  
  
supabase/  
  migrations/  
  functions/  
  seed.sql/  
  config.toml  
  
emails/  
tests/  
e2e/  
docs/
```

Évite :

- les composants de plusieurs centaines de lignes ;

- les appels Supabase dispersés dans les composants visuels ;
- la duplication des règles métier ;
- les types `any` ;
- les statuts représentés par des chaînes arbitraires ;
- la logique d'autorisation uniquement dans l'interface ;
- les transactions financières déclenchées directement depuis le navigateur.

Crée des couches distinctes pour :

- présentation ;
- validation ;
- cas d'usage ;
- accès aux données ;
- autorisation ;
- conformité ;
- paiements ;
- audit ;
- notifications.

7. Direction artistique

L'application doit être particulièrement belle, moderne et interactive.

Elle doit évoquer une marketplace technologique premium et fiable, et non un casino agressif.

Direction visuelle :

- esthétique européenne contemporaine ;
- minimalisme premium ;
- grands espaces ;
- excellente hiérarchie typographique ;
- photographies de lots mises en valeur ;
- animations fluides ;
- micro-interactions précises ;
- ombres légères ;
- dégradés subtils ;
- surfaces translucides utilisées avec parcimonie ;
- coins arrondis cohérents ;
- composants accessibles ;
- excellente lisibilité ;
- qualité identique sur desktop, tablette et mobile.

Évite absolument :

- les effets de machine à sous ;
- les lumières clignotantes ;
- les confettis permanents ;
- les faux comptes à rebours ;
- les mécanismes de pression trompeurs ;
- les dark patterns ;
- les interfaces génériques de template ;

- l'abus de glassmorphism ;
- les animations qui ralentissent la navigation.

Crée deux thèmes :

- thème clair ;
- thème sombre.

Palette suggérée :

- fond clair ivoire ou gris très pâle ;
- texte graphite ;
- couleur principale indigo profond ;
- accent turquoise ou menthe ;
- couleurs d'état accessibles ;
- contraste WCAG AA au minimum.

Crée un design system complet :

- couleurs ;
- typographies ;
- échelles d'espacement ;
- rayons ;
- ombres ;
- breakpoints ;
- grilles ;
- boutons ;
- champs ;
- sélecteurs ;
- onglets ;
- cartes ;
- badges ;
- modales ;
- tiroirs ;
- tableaux ;
- menus ;
- tooltips ;
- toasts ;
- loaders ;
- skeletons ;
- états vides ;
- erreurs ;
- confirmations ;
- graphiques simples ;
- états de vérification ;
- états de campagne.

Ajoute des transitions de page sobres, des hover states précis et des retours visuels immédiats.

Respecte `prefers-reduced-motion`.

8. Internationalisation

Prépare l'application pour plusieurs marchés.

Langues initiales :

- anglais britannique ;
- anglais américain.

Prévois ensuite une extension facile à d'autres langues.

Exigences :

- textes centralisés ;
- formats de date localisés ;
- formats monétaires localisés ;
- fuseaux horaires correctement gérés ;
- stockage des dates en UTC ;
- affichage dans le fuseau pertinent ;
- terminologie adaptable selon le marché ;
- routes localisées si l'architecture retenue le permet proprement ;
- règles officielles associées à une langue et à une version.

Devises initiales :

- GBP ;
- USD.

Ne stocke jamais une somme financière en nombre flottant. Utilise des unités monétaires mineures entières, par exemple `amount_minor`.

9. Pages publiques

9.1 Page d'accueil

Construis une page d'accueil premium comprenant :

- une barre de navigation ;
- une hero section immersive ;
- une proposition de valeur claire ;
- un champ de recherche ;
- des campagnes mises en avant ;
- les catégories populaires ;
- les campagnes qui se terminent prochainement ;
- une explication en trois étapes ;
- une section sur la transparence ;
- une section sur la sécurité ;
- une présentation de la voie gratuite ;
- des témoignages de démonstration explicitement identifiés ;
- une FAQ ;
- un appel à l'action pour les vendeurs ;

- un footer complet.

9.2 Catalogue

Fonctionnalités :

- grille responsive ;
- vue liste ;
- recherche ;
- recherche temporisée ;
- filtres ;
- tri ;
- pagination ou chargement progressif ;
- filtres synchronisés avec l'URL ;
- état de chargement ;
- skeletons ;
- état vide ;
- gestion des erreurs ;
- catégories ;
- type de campagne ;
- valeur du lot ;
- prix d'une entrée ;
- pays d'éligibilité ;
- date de clôture ;
- pourcentage de progression ;
- campagnes disponibles ou terminées.

Ne montre à l'utilisateur que les campagnes visibles dans sa région, sauf lorsqu'un mode explicatif lui indique clairement qu'il n'est pas éligible.

9.3 Fiche campagne

Afficher :

- galerie d'images ;
- zoom ;
- titre ;
- catégorie ;
- description ;
- caractéristiques ;
- valeur déclarée ;
- localisation approximative ;
- identité publique du vendeur ;
- statut de vérification ;
- prix de l'entrée payante, le cas échéant ;
- existence et accès à la voie gratuite ;
- nombre maximal d'entrées ;
- nombre d'entrées attribuées ;
- progression ;
- date de début ;
- date de fin ;
- compte à rebours honnête ;

- règles officielles ;
- restrictions territoriales ;
- âge minimum ;
- méthode de détermination du gagnant ;
- politique de remise du lot ;
- questions et réponses ;
- favoris ;
- partage ;
- campagnes similaires ;
- historique public pertinent ;
- statut du tirage après clôture.

Avant toute entrée, afficher un récapitulatif clair :

- type d'entrée ;
- quantité ;
- prix ;
- chances ou méthode de calcul disponible ;
- restrictions ;
- règles officielles ;
- politique de remboursement ;
- confirmation d'âge et d'éligibilité.

9.4 Pages institutionnelles

Créer :

- How it works ;
- Trust & Safety ;
- Free Entry Route ;
- Responsible Participation ;
- Seller Standards ;
- Prohibited Items ;
- Official Rules ;
- Privacy ;
- Terms ;
- Cookies ;
- Accessibility ;
- Help Centre ;
- Contact ;
- Complaints ;
- Dispute Resolution.

Les contenus juridiques doivent être identifiés comme modèles à faire valider.

10. Authentification

Implémente Supabase Auth avec :

- inscription par email ;
- confirmation de l'email ;

- connexion ;
- déconnexion ;
- mot de passe oublié ;
- réinitialisation ;
- connexion Google facultative ;
- passkeys si leur intégration stable est raisonnable ;
- sessions sécurisées côté serveur ;
- protection des routes ;
- révocation de session ;
- gestion des appareils ou sessions actives ;
- suppression de compte ;
- export des données.

À la création d'un compte, crée automatiquement un profil utilisateur par un mécanisme serveur sécurisé.

Rôles :

- user ;
- seller ;
- moderator ;
- compliance ;
- support ;
- finance ;
- admin ;
- super_admin .

Ne stocke pas l'autorisation critique dans des métadonnées modifiables par l'utilisateur.

11. Profil utilisateur

Créer un espace personnel contenant :

- avatar ;
- nom d'affichage ;
- nom légal privé si nécessaire ;
- biographie ;
- pays ;
- subdivision administrative ;
- ville facultative ;
- date de naissance privée ;
- langue ;
- devise ;
- statut d'identité ;
- statut d'âge ;
- statut du compte ;
- préférences marketing ;
- préférences de notifications ;
- participations ;
- paiements ;
- remboursements ;

- gains ;
- favoris ;
- campagnes suivies ;
- demandes d'assistance ;
- paramètres de sécurité ;
- historique des consentements.

Minimise les données personnelles collectées.

Masque les données sensibles dans les interfaces administratives lorsqu'elles ne sont pas nécessaires.

12. Espace vendeur

Créer un onboarding vendeur comprenant :

- identité ;
- coordonnées ;
- pays ;
- statut individuel ou entreprise ;
- vérification KYC ou KYB ;
- coordonnées de versement ;
- acceptation des conditions vendeur ;
- validation manuelle ;
- statut du compte ;
- limites de publication ;
- catégories autorisées.

Tableau de bord vendeur :

- campagnes ;
- brouillons ;
- performances ;
- participations ;
- revenus estimés ;
- paiements en attente ;
- remboursements ;
- questions ;
- documents ;
- modération ;
- litiges ;
- notifications ;
- paramètres.

Les chiffres financiers doivent distinguer :

- montant brut ;
- frais du prestataire ;
- commission de la plateforme ;
- taxes ;
- réserves ;
- remboursements ;

- montant net estimé ;
- montant versé.

13. Création d'une campagne

Crée un assistant multiétape avec sauvegarde automatique.

Étapes :

1. type de campagne ;
2. catégorie ;
3. territoire proposé ;
4. informations sur le lot ;
5. description ;
6. images ;
7. justificatifs de propriété ;
8. valeur du lot ;
9. mécanique de participation ;
10. prix et nombre maximal d'entrées ;
11. voie gratuite alternative ;
12. dates ;
13. modalités de remise ;
14. règles officielles ;
15. aperçu ;
16. soumission à la modération.

Exigences :

- brouillons enregistrés ;
- reprise ultérieure ;
- validation Zod ;
- validation serveur ;
- autosave temporisé ;
- galerie réordonnable ;
- texte alternatif ;
- image de couverture ;
- compression client raisonnable ;
- vérification serveur du type MIME ;
- limite de taille ;
- noms de fichiers non prévisibles ;
- documents privés ;
- images de brouillon privées ;
- suppression des fichiers orphelins ;
- contrôle des extensions ;
- préparation à une analyse antivirus ;
- modération des images et du texte derrière des interfaces de service.

Le vendeur ne choisit jamais librement des règles interdites. Les choix disponibles doivent être dérivés du moteur de conformité.

14. Types de campagnes

Modélise au minimum :

- `paid_prize_competition` ;
- `free_draw` ;
- `sweepstakes` ;
- `hybrid_paid_with_free_route` ;
- `skill_based_competition` .

Chaque type doit disposer de règles configurables.

Pour une compétition fondée sur l'habileté :

- conserver la question ou l'épreuve ;
- conserver les réponses possibles ;
- conserver la réponse correcte ou la grille d'évaluation de manière sécurisée ;
- empêcher sa divulgation au navigateur avant validation ;
- conserver la version de l'épreuve ;
- enregistrer le résultat de l'utilisateur ;
- assurer une procédure cohérente en cas d'égalité.

Pour une voie gratuite :

- afficher des instructions accessibles ;
- enregistrer les demandes reçues ;
- appliquer des règles équivalentes prévues par la configuration ;
- conserver la preuve de traitement ;
- empêcher que la voie gratuite soit artificiellement cachée ou rendue inutilisable ;
- permettre plusieurs modalités configurables, par exemple formulaire gratuit ou courrier, selon la juridiction et les règles approuvées.

15. Moteur de conformité territoriale

Construis un moteur de règles piloté par la base de données.

Il doit répondre à des questions telles que :

- ce vendeur peut-il créer cette campagne ?
- ce lot est-il autorisé ?
- cette mécanique est-elle autorisée ?
- cette campagne peut-elle être montrée à cet utilisateur ?
- cet utilisateur peut-il obtenir une entrée ?
- une entrée payante est-elle autorisée ?
- une voie gratuite est-elle obligatoire ?
- une question d'habileté est-elle obligatoire ?
- quel âge minimum s'applique ?
- un contrôle d'identité est-il requis ?
- quelles limites financières s'appliquent ?
- quels textes doivent être affichés ?
- quel prestataire de paiement peut être utilisé ?

- quelles règles officielles s'appliquent ?

Hiérarchie territoriale :

- global ;
- pays ;
- subdivision administrative ;
- éventuellement territoire postal ou zone exclue.

Crée des tables configurables pour :

- juridictions ;
- subdivisions ;
- formats autorisés ;
- catégories autorisées ;
- catégories interdites ;
- âge minimum ;
- exigences KYC ;
- exigences KYB ;
- plafonds ;
- voie gratuite obligatoire ;
- géolocalisation obligatoire ;
- prestataires de paiement autorisés ;
- textes obligatoires ;
- versions des règles ;
- dates d'entrée en vigueur ;
- dates d'expiration ;
- statut d'approbation juridique.

Toute modification réglementaire doit être auditée et versionnée.

Adopte une politique de refus par défaut : si aucune règle approuvée ne permet explicitement une opération, bloque-la.

N'invente pas les règles légales définitives. Fournis des données initiales prudentes marquées

`requires_legal_approval=true` .

16. Géolocalisation et résidence

Distingue :

- adresse déclarée ;
- résidence vérifiée ;
- position IP ;
- pays du moyen de paiement ;
- pays du document d'identité ;
- adresse de facturation.

Crée une interface `GeolocationProvider` .

Prévois :

- détection IP côté serveur ;
- stockage minimal ;
- hash ou réduction de précision lorsque possible ;
- détection des incohérences ;
- blocage ou examen manuel ;
- gestion d'un VPN suspect ;
- possibilité de demander une preuve de résidence ;
- aucune confiance exclusive dans la géolocalisation du navigateur.

L'adresse IP ne doit pas être conservée en clair dans les journaux applicatifs ordinaires.

17. Vérification d'identité et d'âge

Crée une interface `IdentityVerificationProvider`.

États possibles :

- `not_started` ;
- `pending` ;
- `verified` ;
- `failed` ;
- `manual_review` ;
- `expired` .

Ne stocke pas directement les documents d'identité lorsque le fournisseur KYC peut les conserver.

Stocke uniquement les références, résultats nécessaires, dates, statut et données minimales autorisées.

Exige une nouvelle vérification lorsque les règles configurées le demandent.

18. Participations et tickets

La plateforme doit pouvoir autoriser plusieurs tickets par utilisateur lorsque la campagne et la juridiction l'autorisent.

Ne limite donc pas globalement à un ticket par personne.

Chaque campagne doit définir :

- minimum par transaction ;
- maximum par transaction ;
- maximum par utilisateur ;
- maximum total ;
- prix unitaire ;
- éventuels packs autorisés ;
- voie gratuite ;
- date d'ouverture ;
- date de fermeture ;

- critères d'éligibilité.

Une entrée ne doit être confirmée qu'après :

- validation territoriale ;
- validation de l'âge ;
- validation du statut du compte ;
- validation de la campagne ;
- validation des plafonds ;
- validation du paiement, le cas échéant ;
- vérification d'idempotence.

Utilise des transactions PostgreSQL pour éviter :

- les dépassements de stock ;
- les doubles entrées ;
- les doubles débits ;
- les incohérences de compteurs ;
- les courses concurrentes.

Ne calcule pas les compteurs critiques uniquement avec des mises à jour client.

19. Paiements

Crée une abstraction indépendante :

```
interface PaymentProvider {
    createCustomer(...)
    createPaymentIntent(...)
    confirmPayment(...)
    cancelPayment(...)
    refundPayment(...)
    createSellerAccount(...)
    getSellerAccountStatus(...)
    createPayout(...)
    verifyWebhook(...)
    parseWebhook(...)
}
```

Implémente :

- un `MockPaymentProvider` complet pour le développement et les tests ;
- une structure prête pour un premier fournisseur réel ;
- une configuration des fournisseurs par juridiction ;
- aucun fournisseur réel activé sans variables d'environnement explicites ;
- aucune affirmation selon laquelle un fournisseur accepte l'activité sans approbation contractuelle.

Le fournisseur réel doit être remplaçable.

Exigences :

- webhooks signés ;
- idempotence ;
- journal des événements ;
- table d'événements reçus ;
- protection contre les replays ;
- traitements asynchrones ;
- retries contrôlés ;
- machine à états de paiement ;
- rapprochement ;
- remboursements complets et partiels ;
- échecs ;
- litiges ;
- chargebacks ;
- réserves ;
- versements vendeur ;
- blocage des fonds jusqu'aux conditions de libération ;
- aucune manipulation de données de carte par l'application ;
- aucun secret dans le frontend.

Statuts de paiement :

- `created` ;
- `requires_action` ;
- `processing` ;
- `succeeded` ;
- `failed` ;
- `cancelled` ;
- `partially_refunded` ;
- `refunded` ;
- `disputed` .

Statuts de versement :

- `pending` ;
- `held` ;
- `approved` ;
- `processing` ;
- `paid` ;
- `failed` ;
- `reversed` .

20. Clôture et sélection du gagnant

Construis un processus serveur robuste.

Étapes :

1. fermer les nouvelles participations ;
2. attendre la finalisation des paiements en cours ;

3. annuler ou exclure les entrées invalides selon les règles ;
4. figer la liste des entrées éligibles ;
5. produire un snapshot canonique ;
6. calculer son hash ;
7. enregistrer la version des règles ;
8. sélectionner le gagnant avec une source aléatoire cryptographiquement sûre ou le mécanisme approuvé ;
9. conserver les données d'audit ;
10. vérifier l'éligibilité du gagnant ;
11. gérer un gagnant inéligible selon les règles officielles ;
12. notifier les parties ;
13. déclencher le processus de remise du lot ;
14. publier un résultat respectueux de la vie privée.

Contraintes :

- opération côté serveur uniquement ;
- aucun `Math.random()` ;
- aucune sélection dans le navigateur ;
- impossible de relancer silencieusement ;
- impossible de modifier le snapshot après le tirage ;
- procédure de reroll strictement encadrée ;
- raison obligatoire ;
- autorisation renforcée ;
- double validation administrative pour une nouvelle sélection ;
- journal d'audit immuable ;
- identifiant public du tirage ;
- page publique de vérification ;
- données personnelles masquées.

Évalue l'utilisation d'une graine publique ou d'une source externe vérifiable, mais n'en fais pas une fausse promesse de conformité.

21. Remise des lots

Modélise le processus :

- gagnant provisoire ;
- vérification ;
- acceptation ;
- coordonnées ;
- livraison ;
- collecte ;
- transfert de propriété ;
- preuve de réception ;
- refus ;
- expiration ;
- litige ;
- remplacement du gagnant selon les règles.

Pour les véhicules et biens immobiliers, crée seulement l'architecture et les workflows documentaires nécessaires. Ne prétends pas automatiser les transferts légaux propres à chaque territoire sans intégration et validation adaptées.

22. Modération et objets interdits

Crée une politique de catégories interdites configurable.

Prévois notamment le blocage ou l'examen renforcé de :

- armes ;
- drogues ;
- médicaments réglementés ;
- produits contrefaits ;
- objets volés ;
- espèces protégées ;
- produits dangereux ;
- produits pour adultes soumis à restriction ;
- services illégaux ;
- actifs financiers ;
- produits nécessitant une licence ;
- lots sans preuve de propriété.

Fonctionnalités :

- signalement ;
- file de modération ;
- score de risque ;
- notes internes ;
- demandes de documents ;
- historique ;
- suspension ;
- rejet ;
- appel ;
- actions groupées ;
- séparation entre données publiques et notes internes.

23. Administration

Crée un véritable back-office protégé.

Sections :

- vue d'ensemble ;
- utilisateurs ;
- vendeurs ;
- campagnes ;
- modération ;
- juridictions ;
- règles territoriales ;

- paiements ;
- remboursements ;
- versements ;
- tirages ;
- gagnants ;
- KYC/KYB ;
- signalements ;
- litiges ;
- support ;
- notifications ;
- contenu ;
- catégories ;
- feature flags ;
- audit ;
- santé du système.

Fonctionnalités :

- recherche ;
- filtres ;
- pagination ;
- exports contrôlés ;
- actions avec confirmation ;
- justification obligatoire pour les opérations sensibles ;
- autorisations granulaires ;
- journalisation ;
- double validation pour certaines opérations ;
- masquage des données sensibles ;
- mode lecture seule pour certains rôles.

Ne rends jamais les routes administratives accessibles uniquement grâce à un contrôle visuel.

24. Notifications

Implémente :

- notifications in-app ;
- emails ;
- préférences ;
- statut lu/non lu ;
- Supabase Realtime pour les notifications pertinentes ;
- templates versionnés.

Événements :

- inscription ;
- vérification ;
- campagne soumise ;
- campagne approuvée ;
- modifications demandées ;
- campagne active ;

- entrée confirmée ;
- paiement échoué ;
- remboursement ;
- fin prochaine ;
- campagne clôturée ;
- gagnant provisoire ;
- gagnant confirmé ;
- remise du lot ;
- message du support ;
- litige.

Évite le spam et regroupe les notifications répétitives.

25. Schéma PostgreSQL

Conçois un schéma normalisé incluant au minimum :

Identité

- profiles
- user_private_details
- user_addresses
- user_consent
- user_sessions_metadata
- identity_verifications
- seller_profiles
- seller_verifications

Catalogue

- categories
- campaigns
- campaign_images
- campaign_documents
- campaign_rules_versions
- campaign_eligibility_regions
- campaign_questions
- campaign_answers
- favorites

Conformité

- jurisdictions
- jurisdiction_rules
- jurisdiction_campaign_types
- jurisdiction_categories
- legal_rule_versions
- compliance_decisions
- geo_checks
- risk_flags
- feature_flags

Participations

- entry_orders
- entries
- free_entry_requests
- skill_questions
- skill_question_options
- skill_responses
- entry_reservations

Finance

- payment_customers
- payment_transactions
- payment_events
- refunds
- seller_balances
- payouts
- platform_fees
- financial_ledger

Tirages et remise

- draws
- draw_snapshots
- draw_entries
- winner_verifications
- prize_fulfilments

Exploitation

- notifications
- reports
- disputes
- support_threads
- support_messages
- moderation_cases
- admin_actions
- audit_logs
- webhook_events
- outbox_events

Utilise :

- UUID ;
- contraintes étrangères ;
- contraintes uniques ;
- contraintes `CHECK` ;
- enums PostgreSQL seulement lorsqu'ils ne rendent pas les évolutions inutilement difficiles ;
- index adaptés ;
- index partiels ;

- colonnes `created_at` et `updated_at` ;
- suppression logique lorsqu'elle est nécessaire ;
- fonctions transactionnelles pour les opérations critiques.

Ne duplique pas un ledger financier avec des totaux modifiables sans source comptable.

Le ledger doit être append-only autant que possible.

26. Statuts de campagne

Crée une machine à états contrôlée :

- `draft` ;
- `submitted` ;
- `under_review` ;
- `changes_requested` ;
- `approved` ;
- `scheduled` ;
- `active` ;
- `paused` ;
- `sold_out` ;
- `closing` ;
- `drawing` ;
- `winner_pending` ;
- `winner_confirmed` ;
- `fulfilment` ;
- `completed` ;
- `cancelled` ;
- `rejected` ;
- `disputed` .

Les transitions doivent être validées côté serveur et journalisées.

Interdis les mises à jour arbitraires de statut depuis le client.

27. Sécurité Supabase

Active Row Level Security sur toutes les tables exposées.

Crée et teste explicitement les politiques.

Principes :

- lecture publique limitée aux campagnes publiées ;
- utilisateur limité à ses données ;
- vendeur limité à ses campagnes et informations autorisées ;
- modérateur limité à son périmètre ;
- administrateur contrôlé côté serveur ;
- aucune clé privilégiée dans le navigateur ;
- opérations financières uniquement avec une clé serveur sécurisée ;

- service role uniquement dans un environnement serveur ;
- aucune politique permissive générique ;
- fonctions `SECURITY_DEFINER` rares, auditable et avec `search_path` fixé ;
- révocation des privilèges inutiles ;
- validation des claims ;
- protection des buckets Storage ;
- URLs signées pour les documents privés.

Écris des tests de politiques RLS avec plusieurs identités.

Teste les tentatives d'accès transversal entre utilisateurs.

28. Sécurité applicative

Implémente :

- protections CSRF adaptées à l'architecture ;
- validation Zod côté client et serveur ;
- encodage des sorties ;
- politique CSP ;
- en-têtes de sécurité ;
- limitation de débit ;
- protection anti-bot extensible ;
- CAPTCHA uniquement lorsque nécessaire ;
- idempotency keys ;
- contrôle des uploads ;
- prévention des IDOR ;
- protection contre les injections ;
- protection contre les redirections ouvertes ;
- cookies sécurisés ;
- logs structurés ;
- corrélation des requêtes ;
- masquage des secrets ;
- gestion centralisée des erreurs ;
- pages d'erreur sans fuite d'informations ;
- dépendances auditées ;
- vérification des signatures de webhooks ;
- mécanisme d'outbox pour les événements critiques.

Ajoute un document `docs/THREAT_MODEL.md` avec :

- actifs ;
- acteurs ;
- frontières de confiance ;
- scénarios d'attaque ;
- fraude vendeur ;
- fraude participant ;
- fraude administrateur ;
- compromission de compte ;
- double dépense ;
- manipulation du tirage ;

- contournement géographique ;
- falsification KYC ;
- abus de remboursement ;
- chargebacks ;
- vol de lot ;
- altération des journaux ;
- mesures de réduction du risque.

29. Protection des utilisateurs

Ajoute des fonctionnalités configurables :

- limites de dépense ;
- limites de dépôt ou d'achat ;
- période de pause ;
- auto-exclusion ;
- fermeture du compte ;
- historique des dépenses ;
- rappels d'activité ;
- blocage marketing ;
- restrictions par âge ;
- liens d'aide localisés ;
- blocage des campagnes payantes lorsqu'une limite est atteinte.

Même lorsque ces fonctions ne sont pas juridiquement obligatoires, l'architecture doit pouvoir les prendre en charge.

30. Accessibilité et performance

Respecte WCAG 2.2 AA autant que possible.

Exigences :

- navigation clavier ;
- focus visible ;
- labels ;
- messages d'erreur accessibles ;
- lecteurs d'écran ;
- contraste ;
- alternatives textuelles ;
- modales accessibles ;
- animations réduites ;
- cibles tactiles suffisantes.

Performance :

- images optimisées ;
- dimensions réservées ;
- lazy loading ;
- Server Components ;
- streaming lorsque pertinent ;

- minimisation du JavaScript client ;
- requêtes indexées ;
- pagination ;
- cache raisonné ;
- aucune donnée utilisateur sensible mise en cache publiquement ;
- bons scores Core Web Vitals.

31. SEO

Ajoute :

- métadonnées ;
- titres ;
- descriptions ;
- Open Graph ;
- Twitter Cards ;
- sitemap ;
- robots.txt ;
- canonical URLs ;
- données structurées pertinentes ;
- pages de campagne partageables ;
- gestion des campagnes expirées ;
- redirections propres ;
- pages `not - found` .

Ne publie pas dans les données structurées des informations trompeuses ou non vérifiées.

32. Analytics et observabilité

Crée une abstraction pour l'analytics respectueuse du consentement.

Événements :

- vue de campagne ;
- recherche ;
- filtre ;
- ajout aux favoris ;
- démarrage d'entrée ;
- paiement initié ;
- paiement confirmé ;
- voie gratuite consultée ;
- participation confirmée ;
- création de campagne ;
- soumission ;
- conversion vendeur.

Ajoute :

- logs structurés ;
- gestion des erreurs ;
- traces pour les opérations critiques ;

- health checks ;
- alertes documentées ;
- métriques de webhooks ;
- métriques de paiement ;
- métriques de tirage ;
- métriques de fraude.

Ne journalise pas les secrets, réponses KYC complètes ou données de carte.

33. Données de démonstration

Crée un seed réaliste avec :

- plusieurs utilisateurs ;
- plusieurs vendeurs ;
- catégories ;
- campagnes actives ;
- campagnes terminées ;
- campagnes en modération ;
- participations gratuites et payantes simulées ;
- notifications ;
- questions ;
- paiements fictifs ;
- un tirage fictif auditable ;
- juridictions de démonstration ;
- règles marquées comme non approuvées juridiquement.

Utilise des images placeholders légales ou des dégradés générés localement.

N'utilise aucune marque tierce comme si elle était partenaire.

34. Tests

Tests unitaires

Teste :

- calculs monétaires ;
- éligibilité ;
- limites de tickets ;
- prix ;
- frais ;
- transitions d'état ;
- validations ;
- règles territoriales ;
- génération de hash ;
- idempotence ;
- traitement des webhooks.

Tests d'intégration

Teste :

- création de profil ;
- création de campagne ;
- modération ;
- entrée gratuite ;
- entrée payante simulée ;
- paiement réussi ;
- paiement échoué ;
- remboursement ;
- clôture ;
- snapshot ;
- sélection ;
- vérification du gagnant ;
- versement simulé ;
- politiques RLS.

Tests Playwright

Couvre au minimum :

1. inscription ;
2. connexion ;
3. navigation du catalogue ;
4. filtres ;
5. fiche campagne ;
6. entrée gratuite ;
7. achat simulé ;
8. création vendeur ;
9. création d'une campagne ;
10. soumission ;
11. approbation administrateur ;
12. clôture ;
13. tirage ;
14. consultation du résultat ;
15. remboursement ;
16. accès interdit à un autre utilisateur ;
17. accès administrateur interdit à un utilisateur standard.

Teste desktop et mobile.

35. Intégration continue

Crée un workflow GitHub Actions exécutant :

- installation ;
- lint ;
- format check ;
- typecheck ;

- tests unitaires ;
- tests d'intégration disponibles ;
- build ;
- tests Playwright dans un environnement adapté ;
- vérification des migrations ;
- audit raisonnable des dépendances.

Le pipeline doit échouer en cas d'erreur.

36. Variables d'environnement

Crée `.env.example` avec des noms explicites, sans secret réel.

Inclure notamment :

```

NEXT_PUBLIC_APP_URL=
NEXT_PUBLIC_APP_NAME=

NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SECRET_KEY=
SUPABASE_DATABASE_URL=

PAYMENT_PROVIDER=
PAYMENT_PROVIDER_API_KEY=
PAYMENT_WEBHOOK_SECRET=

EMAIL_PROVIDER=
EMAIL_PROVIDER_API_KEY=
EMAIL_FROM=

GEOLOCATION_PROVIDER=
GEOLOCATION_API_KEY=

KYC_PROVIDER=
KYC_API_KEY=
KYC_WEBHOOK_SECRET=

APP_ENCRYPTION_KEY=
CRON_SECRET=
INTERNAL_API_SECRET=

ENABLE_REAL_PAYMENTS=false
ENABLE_PAID_ENTRIES=false
ENABLE_FREE_ENTRIES=true
ENABLE_SKILL_COMPETITIONS=true

```

Les flags ne remplacent pas le moteur de conformité. Même activée globalement, une fonction doit rester bloquée dans une juridiction non approuvée.

37. Documentation

Crée :

- README.md
- docs/ARCHITECTURE.md
- docs/DATABASE.md
- docs/RLS.md
- docs/SECURITY.md
- docs/THREAT_MODEL.md
- docs/PAYMENTS.md
- docs/COMPLIANCE_ENGINE.md
- docs/DRAWS.md
- docs/DEPLOYMENT.md
- docs/OPERATIONS.md
- docs/INCIDENT_RESPONSE.md
- docs/LEGAL_REVIEW_CHECKLIST.md
- docs/PROVIDER_APPROVAL_CHECKLIST.md
- docs/TESTING.md

Le README doit contenir :

- prérequis ;
- installation ;
- configuration Supabase locale ;
- migrations ;
- seed ;
- lancement ;
- tests ;
- build ;
- déploiement ;
- comptes de démonstration ;
- architecture résumée.

La checklist juridique doit demander au minimum une validation par territoire de :

- type de promotion ;
- voie gratuite ;
- niveau réel de compétence requis ;
- règles officielles ;
- âge ;
- résidence ;
- obligations d'enregistrement ;
- cautions ou dépôts éventuels ;
- obligations fiscales ;
- publicité ;
- traitement des données ;
- KYC/AML ;
- remise du lot ;
- biens immobiliers ;
- véhicules ;

- restrictions postales ;
- restrictions des prestataires ;
- protection des consommateurs ;
- politique de remboursement.

38. Critères d'acceptation

Le projet n'est considéré comme terminé que si :

- l'application démarre localement ;
- la base peut être créée à partir des migrations ;
- le seed fonctionne ;
- l'inscription fonctionne ;
- la connexion fonctionne ;
- le catalogue fonctionne ;
- les campagnes sont filtrées par territoire ;
- un vendeur peut créer un brouillon ;
- une campagne peut être modérée ;
- une participation gratuite fonctionne ;
- un achat simulé fonctionne ;
- les webhooks simulés fonctionnent ;
- les limites d'entrées résistent aux requêtes concurrentes ;
- la clôture fonctionne ;
- le snapshot est créé ;
- le tirage serveur fonctionne ;
- le résultat est auditable ;
- le back-office est protégé ;
- les politiques RLS sont testées ;
- les tests critiques passent ;
- le build de production passe ;
- le design est réellement abouti sur mobile et desktop ;
- la documentation permet à un autre développeur de reprendre le projet.

39. Ordre de réalisation

Procède dans cet ordre :

Phase 1 — Audit du dépôt et architecture.

Phase 2 — Initialisation Next.js, qualité et design system.

Phase 3 — Supabase local, schéma, migrations et seed.

Phase 4 — Authentification et profils.

Phase 5 — Catalogue et fiches campagnes.

Phase 6 — Espace vendeur et création de campagne.

Phase 7 — Moteur territorial et éligibilité.

Phase 8 — Participations gratuites et compétitions d'habileté.

Phase 9 — Abstraction de paiement et fournisseur simulé.

Phase 10 — Paiements, ledger, remboursements et webhooks simulés.

Phase 11 — Clôture, snapshot, tirage et gagnant.

Phase 12 — Administration et modération.

Phase 13 — Notifications, emails et temps réel.

Phase 14 — Sécurité, accessibilité et performance.

Phase 15 — Tests complets et CI.

Phase 16 — Documentation et audit final.

40. Rapport final obligatoire

Lorsque l'implémentation est terminée, fournis un rapport final contenant :

1. ce qui a été développé ;
2. l'architecture retenue ;
3. les principaux choix techniques ;
4. les migrations créées ;
5. les politiques RLS créées ;
6. les tests exécutés ;
7. les résultats exacts du lint, du typecheck, des tests et du build ;
8. les commandes de lancement ;
9. les variables à configurer ;
10. les étapes de déploiement ;
11. les fournisseurs externes restant à sélectionner ;
12. les fonctions désactivées par défaut ;
13. les validations juridiques encore nécessaires ;
14. les risques résiduels ;
15. les améliorations recommandées après le MVP.

Commence maintenant par inspecter le dépôt et créer le plan d'architecture. Ensuite, développe l'application phase par phase sans t'arrêter après la conception visuelle.